



Coding Practice Set — Brute Force & Optimal Solutions

This set documents **7 coding questions** from the uploaded practice file, each with a **brute-force** and an **optimal** Python solution, written for interview/training use.

1. Integer Square Root

Given a non-negative integer, return **floor(sqrt(n))** without using **sqrt**, **pow**, etc.

Brute Force — $O(\sqrt{n})$

```
def integer_sqrt_brute(n):
    if n < 0:
        return -1

    i = 0

    while (i + 1) * (i + 1) <= n:
        i += 1

    return i
```

Optimal — Binary Search — $O(\log n)$

```
def integer_sqrt(n):
    if n < 0:
        return -1

    if n < 2:
        return n

    left, right = 1, n // 2
    answer = 1

    while left <= right:
        mid = (left + right) // 2

        if mid * mid <= n:
            answer = mid
            left = mid + 1
        else:
            right = mid - 1

    return answer
```



2. Pairs With Sum Less Than K

The array is sorted and we need unique index pairs (i, j) where $i < j$ and $\text{arr}[i] + \text{arr}[j] < k$. The examples confirm that $(0,1)$ and $(1,0)$ are considered the same.

Brute Force — $O(n^2)$

```
def pairs_brute(arr, k):
    result = []

    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] + arr[j] < k:
                result.append((i, j))

    return result
```

Optimal — Two Pointers — $O(n)$

Because the array is already sorted:

```
def pairs_optimal(arr, k):
    result = []

    left = 0
    right = len(arr) - 1

    while left < right:
        if arr[left] + arr[right] < k:
            # Every index from left+1 to right
            # will form a valid pair with left.
            for j in range(left + 1, right + 1):
                result.append((left, j))

            left += 1
        else:
            right -= 1

    return result
```

For $\text{arr} = [1, 2, 3, 4, 5]$, $k = 6$, output: $[(0,1), (0,2), (0,3), (1,2)]$

Important: The output itself can contain $O(n^2)$ pairs, so although the pointer logic is $O(n)$, producing all pairs is $O(n^2)$ in the worst case.



3. Longest Palindromic Path in a Grid

The path starts at (0,0), ends at (m-1,n-1), and can only move **right/down**. The file's examples show that multiple palindromic paths can exist. Since every top-left → bottom-right path has exactly $m + n - 1$ characters, the “longest” valid palindrome is simply a **full-length palindromic path**.

Brute Force — Generate Every Path

```
def longest_palindromic_path_brute(grid):
    m = len(grid)
    n = len(grid[0])

    paths = []

    def dfs(r, c, path):
        if r == m - 1 and c == n - 1:
            s = ''.join(path)
            if s == s[::-1]:
                paths.append(s)
            return

        if r + 1 < m:
            path.append(grid[r + 1][c])
            dfs(r + 1, c, path)
            path.pop()

        if c + 1 < n:
            path.append(grid[r][c + 1])
            dfs(r, c + 1, path)
            path.pop()

    dfs(0, 0, [grid[0][0]])

    if not paths:
        return []

    max_len = max(map(len, paths))

    return [p for p in paths if len(p) == max_len]
```

This is exponential because the number of paths is $C(m+n-2, m-1)$.



Optimal — DP With Two Ends

Instead of generating complete paths, simultaneously move one pointer from the top-left and one pointer from the bottom-right. The characters must match.

```
from functools import lru_cache

def longest_palindromic_path(grid):
    m = len(grid)
    n = len(grid[0])

    @lru_cache(None)
    def dp(r1, c1, r2, c2):
        # Characters at both ends must match
        if grid[r1][c1] != grid[r2][c2]:
            return None

        # They have met
        if (r1, c1) == (r2, c2):
            return grid[r1][c1]

        # Adjacent cells
        if abs(r1 - r2) + abs(c1 - c2) == 1:
            return grid[r1][c1] + grid[r2][c2]

        candidates = []

        # First pointer moves DOWN, second moves UP
        if r1 + 1 < m and r2 - 1 >= 0:
            result = dp(r1 + 1, c1, r2 - 1, c2)
            if result:
                candidates.append(result)

        # First pointer moves DOWN, second moves LEFT
        if r1 + 1 < m and c2 - 1 >= 0:
            result = dp(r1 + 1, c1, r2, c2 - 1)
            if result:
                candidates.append(result)

        # First pointer moves RIGHT, second moves UP
        if c1 + 1 < n and r2 - 1 >= 0:
            result = dp(r1, c1 + 1, r2 - 1, c2)
            if result:
                candidates.append(result)

        # First pointer moves RIGHT, second moves LEFT
        if c1 + 1 < n and c2 - 1 >= 0:
            result = dp(r1, c1 + 1, r2, c2 - 1)
            if result:
                candidates.append(result)

        if not candidates:
            return None

        best = max(candidates, key=len)

        return grid[r1][c1] + best + grid[r2][c2]

    result = dp(0, 0, m - 1, n - 1)
    return [result] if result else []
```

Complexity: approximately $O(m^2n^2)$ states.



4. Minimum Window Substring Containing T as a Subsequence

This differs from the normal Minimum Window Substring problem: **T** must occur as a **subsequence**, not necessarily as a contiguous substring.

Example: S = "abcdebddde", T = "bde" → answer: "bcde" or "bdde"

Brute Force

Generate every substring and check whether T is a subsequence.

```
def min_window_brute(s, t):
    n = len(s)
    m = len(t)

    best = ""

    for left in range(n):
        for right in range(left, n):

            i = left
            j = 0

            while i <= right and j < m:
                if s[i] == t[j]:
                    j += 1

                i += 1

            if j == m:
                current = s[left:right + 1]

                if best == "" or len(current) < len(best):
                    best = current

    return best
```

Optimal — DP — $O(n \times m)$

dp[j] stores the starting position of a substring that can currently form $t[:j+1]$ as a subsequence.

```
def min_window_optimal(s, t):
    n = len(s)
    m = len(t)

    if m == 0:
        return ""

    dp = [-1] * m

    best_start = -1
    best_len = float('inf')

    for i in range(n):

        # Traverse backwards so previous values
        # are not overwritten too early.
        for j in range(m - 1, -1, -1):

            if s[i] == t[j]:

                if j == 0:
                    dp[j] = i
                elif dp[j - 1] != -1:
                    dp[j] = dp[j - 1]

            if dp[m - 1] != -1:
                start = dp[m - 1]
                length = i - start + 1

                if length < best_len:
                    best_len = length
                    best_start = start

    if best_start == -1:
        return ""

    return s[best_start:best_start + best_len]
```



5. Maximum Survey Score

This problem reduces to: (1) find connected components of equal non-zero values, (2) each component becomes a **field**, (3) connect neighbouring fields, (4) a field of value x can move only to a neighbouring field of value $x+1$, (5) find the maximum total number of cells along a route.



Brute Force

First create fields using DFS/BFS, then explore every possible route.

```
def max_survey_brute(grid):
    m = len(grid)
    n = len(grid[0])

    visited = [[False] * n for _ in range(m)]
    fields = []

    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    # Build connected components
    for r in range(m):
        for c in range(n):
            if grid[r][c] == 0 or visited[r][c]:
                continue

            value = grid[r][c]
            cells = []

            stack = [(r, c)]
            visited[r][c] = True

            while stack:
                x, y = stack.pop()
                cells.append((x, y))

                for dx, dy in directions:
                    nx = x + dx
                    ny = y + dy

                    if (0 <= nx < m and 0 <= ny < n and
                        not visited[nx][ny] and grid[nx][ny] == value):
                        visited[nx][ny] = True
                        stack.append((nx, ny))

            fields.append({
                "value": value,
                "cells": cells,
                "size": len(cells),
                "next": []
            })

    # Find neighbouring fields
    cell_to_field = {}

    for idx, field in enumerate(fields):
        for cell in field["cells"]:
            cell_to_field[cell] = idx

    for idx, field in enumerate(fields):
        next_fields = set()

        for r, c in field["cells"]:
            for dr, dc in directions:
                nr, nc = r + dr, c + dc

                if (nr, nc) in cell_to_field:
                    j = cell_to_field[(nr, nc)]

                    if fields[j]["value"] == field["value"] + 1:
                        next_fields.add(j)

        field["next"] = list(next_fields)

    # DFS every possible route
    def dfs(index):
        best = 0

        for nxt in fields[index]["next"]:
            best = max(best, dfs(nxt))

        return fields[index]["size"] + best

    answer = 0

    for i, field in enumerate(fields):
        if field["value"] == 1:
            answer = max(answer, dfs(i))

    return answer
```



Optimal — Memoized DFS / DAG DP

Because movement is strictly $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow \dots$ there cannot be a cycle.

```
from functools import lru_cache

def max_survey_optimal(grid):
    m = len(grid)
    n = len(grid[0])

    visited = [[False] * n for _ in range(m)]
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    fields = []

    # Step 1: Build fields
    for r in range(m):
        for c in range(n):
            if grid[r][c] == 0 or visited[r][c]:
                continue

            value = grid[r][c]
            cells = []

            stack = [(r, c)]
            visited[r][c] = True

            while stack:
                x, y = stack.pop()
                cells.append((x, y))

                for dx, dy in directions:
                    nx, ny = x + dx, y + dy

                    if (0 <= nx < m and 0 <= ny < n and
                        not visited[nx][ny] and grid[nx][ny] == value):
                        visited[nx][ny] = True
                        stack.append((nx, ny))

            fields.append((value, cells))

    # Step 2: Map cell -> field
    cell_to_field = {}

    for i, (_, cells) in enumerate(fields):
        for cell in cells:
            cell_to_field[cell] = i

    # Step 3: Build graph
    graph = [[] for _ in fields]

    for i, (value, cells) in enumerate(fields):
        neighbours = set()

        for r, c in cells:
            for dr, dc in directions:
                nr, nc = r + dr, c + dc

                j = cell_to_field.get((nr, nc))

                if j is not None and fields[j][0] == value + 1:
                    neighbours.add(j)

        graph[i] = list(neighbours)

    # Step 4: DP
    @lru_cache(None)
    def dfs(field_id):
        size = len(fields[field_id][1])
        best_next = 0

        for nxt in graph[field_id]:
            best_next = max(best_next, dfs(nxt))

        return size + best_next

    answer = 0

    for i, (value, _) in enumerate(fields):
        if value == 1:
            answer = max(answer, dfs(i))

    return answer
```



*This is the approach worth teaching for this question because it combines: **Grid DFS** → **Connected Components** → **Graph** → **DAG DP**.*



6. Two Conveyor Belts → Palindromic Interleaving

Two sequences X and Y are given. At every step, we choose the front element of either sequence. The resulting sequence must be a palindrome, and we minimize the number of switches between X and Y. Ties are resolved with $X < Y$. This is the most difficult problem in the set.

Brute Force

Generate every possible interleaving — explores $C(n+m, n)$ possibilities.

```
def conveyor_brute(x, y):
    best = None
    best_switches = float('inf')

    def dfs(i, j, sequence, choices):
        nonlocal best, best_switches

        if i == len(x) and j == len(y):
            s = ''.join(sequence)

            if s != s[::-1]:
                return

            switches = 0
            for k in range(1, len(choices)):
                if choices[k] != choices[k - 1]:
                    switches += 1

            choice_string = ''.join(choices)

            if (switches < best_switches or
                (switches == best_switches and
                 (best is None or choice_string < best))):
                best_switches = switches
                best = choice_string

            return

        if i < len(x):
            sequence.append(x[i]); choices.append('X')
            dfs(i + 1, j, sequence, choices)
            choices.pop(); sequence.pop()

        if j < len(y):
            sequence.append(y[j]); choices.append('Y')
            dfs(i, j + 1, sequence, choices)
            choices.pop(); sequence.pop()

    dfs(0, 0, [], [])

    return best if best is not None else -1
```

Optimal Direction

A normal 2D interleaving DP is **not sufficient** here, because three constraints apply simultaneously: order within X, order within Y, and the final string must be a palindrome, while minimizing switches. The right formulation is a **two-ended DP**, whose state tracks the left position in X and Y, the right position in X and Y, and the last source used on each side — only permitting a transition when the left and right characters match, while minimizing the switch count. For interview purposes this is best treated as an **advanced DP problem**; the brute-force version above is a correct baseline, and the optimized version needs careful state and reconstruction handling.

7. Minimum Number of Intervals to Remove

Remove the minimum number of intervals so the remaining intervals don't overlap — the classic Non-overlapping Intervals problem. [1,2] and [2,3] do not overlap.



Brute Force — $O(2^n)$

Try keeping/removing every interval.

```
def erase_overlap_brute(intervals):
    n = len(intervals)

    def dfs(index, prev_end):
        if index == n:
            return 0

        # Option 1: remove current interval
        remove = 1 + dfs(index + 1, prev_end)

        # Option 2: keep current interval
        keep = float('inf')

        if intervals[index][0] >= prev_end:
            keep = dfs(index + 1, intervals[index][1])

        return min(remove, keep)

    intervals.sort()

    return dfs(0, float('-inf'))
```

Optimal — Greedy — $O(n \log n)$

Key idea: when two intervals overlap, keep the one that finishes earlier — that leaves maximum room for future intervals.

```
def erase_overlap_optimal(intervals):
    if not intervals:
        return 0

    intervals.sort(key=lambda x: x[1])

    removals = 0
    prev_end = intervals[0][1]

    for i in range(1, len(intervals)):
        start, end = intervals[i]

        if start < prev_end:
            removals += 1
        else:
            prev_end = end

    return removals
```

Example: intervals = `[[1,2],[2,4],[4,11],[1,3]]` → output: **1**

Summary

Q	Problem	Brute Force	Optimal
1	Integer Square Root	Linear search	Binary Search
2	Pair Sum < K	Two loops	Two Pointers
3	Palindromic Grid Path	Enumerate paths	Two-ended DP
4	Min Window Subsequence	All substrings	DP
5	Maximum Survey Score	DFS all routes	Connected Components + DAG DP
6	Palindromic Interleaving	All interleavings	Two-ended DP
7	Minimum Interval Removal	Subset/DFS	Greedy by earliest finish

For a Zoho-style interview preparation set, the strongest progression is **Q1 → Q2 → Q7 → Q4 → Q5 → Q3 → Q6**: it moves from basic search/two-pointer/greedy into substantially harder DP and state-space problems.